ICS 604
Applied Data Science

Department of Information and Computer Sciences
University of Hawai`i at Mānoa

Kyungim Baek

Homework Assignment 4

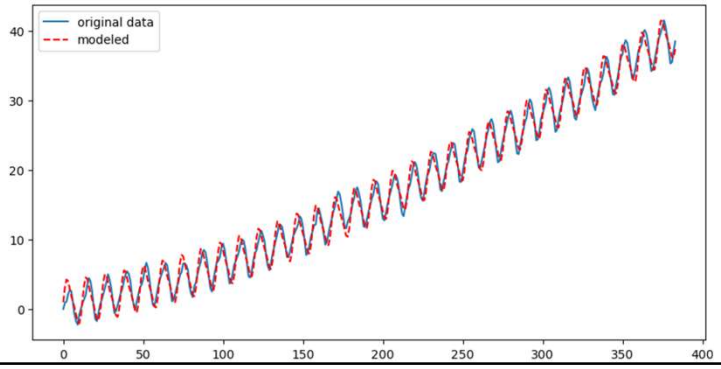
- Due: **11:59 PM** on **Monday, April 27**
- Please carefully read and follow the assignment instructions and problem requirements.
- Submit only the Jupyter Notebook file (.ipynb) to Lamaku.
  - Do not upload the data file.
  - Make sure to **rename your notebook file** as instructed.

Lecture 23

- Residual autocorrelation analysis
- Exponential Smoothing

Previously… Regression-based modeling

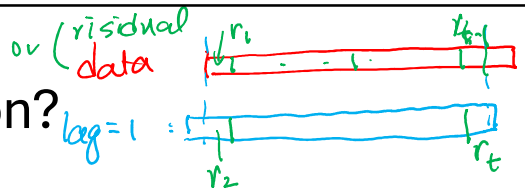
- Power law trend to capture the long-term increase
- 12-month seasonality to model the yearly cycle
- Second harmonic (6-month cycle) to refine and better capture the seasonal fluctuation



Residual autocorrelation analysis

- **Goal:** Evaluate whether our model has captured all structure in the data
- After modeling trend and seasonality, we analyze the residuals.
- Ideally, residuals should behave like white noise:
  - No patterns
  - Constant variance
  - No correlation across time
- If residuals are correlated across time, the model is **missing structure.**

What is autocorrelation?



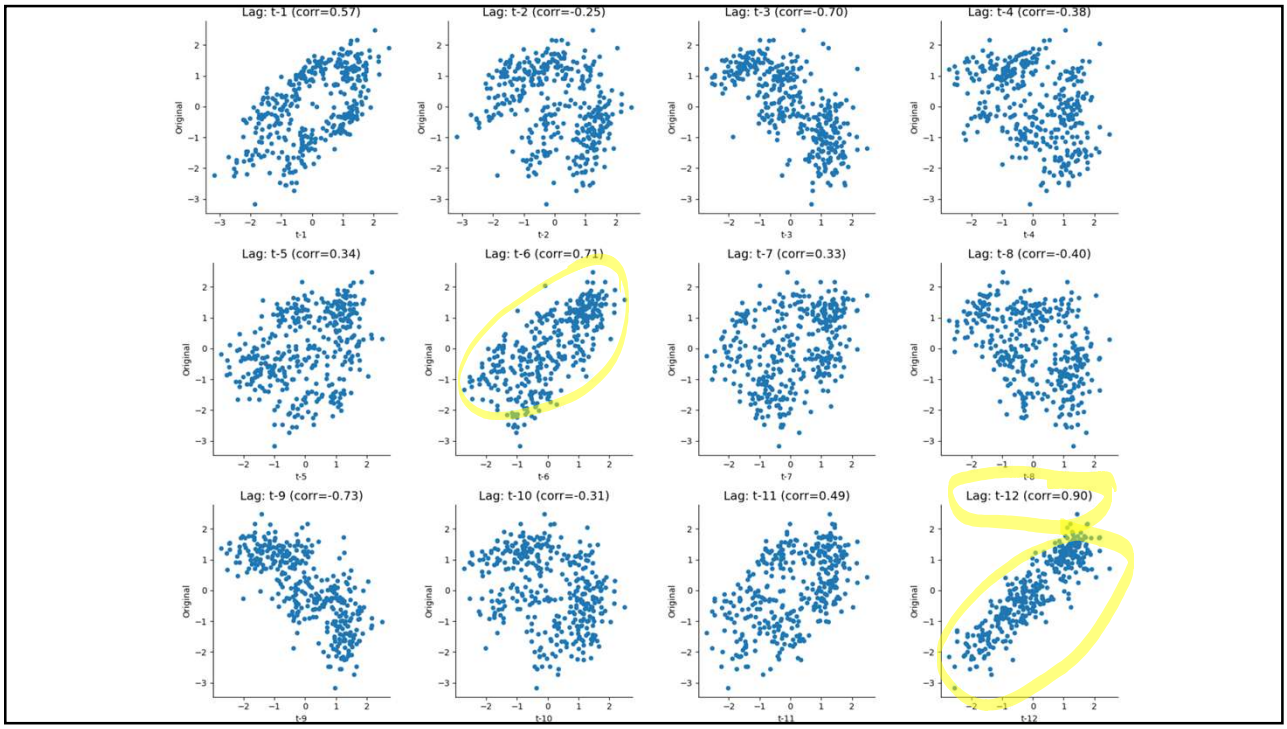
- Autocorrelation measures the relationship between a time series and its past values (lags).
- For our example,
  - For each lag $k = 1, 2, \ldots, 12$:
    - Shift the time series r_t by k time steps to obtain r_{t-k}.
    - Compute the correlation coefficient between the original series r_t and the lagged series r_{t-k}
- We can visualize this using:
  - Scatter plots (residual vs. lagged residual)
  - Correlation values for each lag

```
lags = 12
ncols = 4
nrows = int(np.ceil(lags/ncols))

fig, axes = plt.subplots(ncols=ncols, nrows=nrows, figsize=(4*ncols, 4*nrows))
residuals.columns = ['co2_val']

corr_vals = []
for ax, lag in zip(axes.flat, np.arange(1, lags+1, 1)):
    lag_str = f't-{lag}'
    X = (pd.concat([residuals['co2_val'], residuals['co2_val'].shift(-lag)], axis=1,
                   keys=['y'] + [lag_str]).dropna())

    X.plot(ax=ax, kind='scatter', y='y', x=lag_str)
    corr = X.corr().values[0][1]
    corr_vals.append(corr)
    ax.set_ylabel('Original')
    ax.set_title(f'Lag: {lag_str} (corr={corr:.2f})', fontsize=14)
    ax.set_aspect('equal')
    sns.despine()

fig.tight_layout()
```

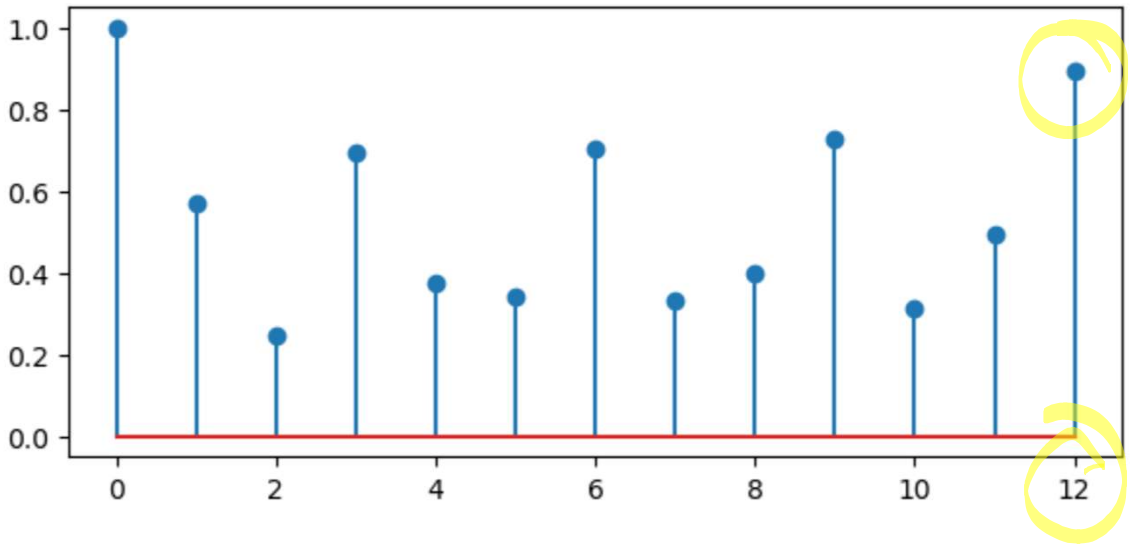
```
display(X.corr())
X.head()
```

	y	t-12
y	1.000000	0.895635
t-12	0.895635	1.000000

	y	t-12
0	-0.977029	-0.379644
1	-2.233396	-2.020294
2	-3.173360	-2.540603
3	-1.870783	-0.876299
4	-0.266840	1.140833




```
plt.figure(figsize=(7, 3))
plt.stem(np.arange(len(corr_vals)), np.abs(corr_vals));
```

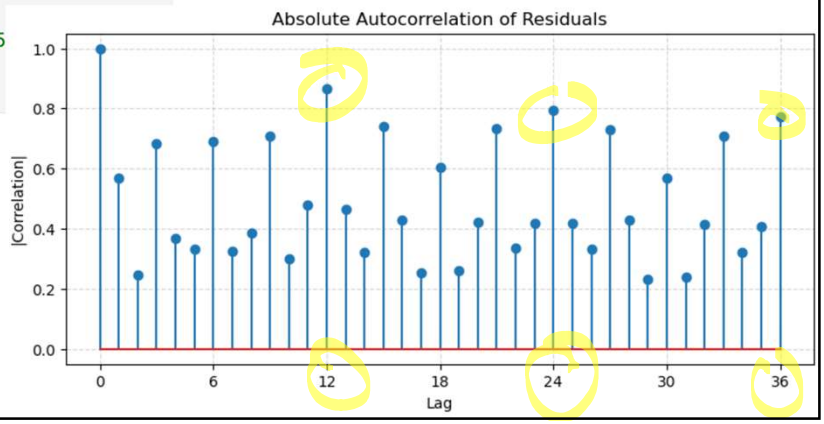


Results from our example

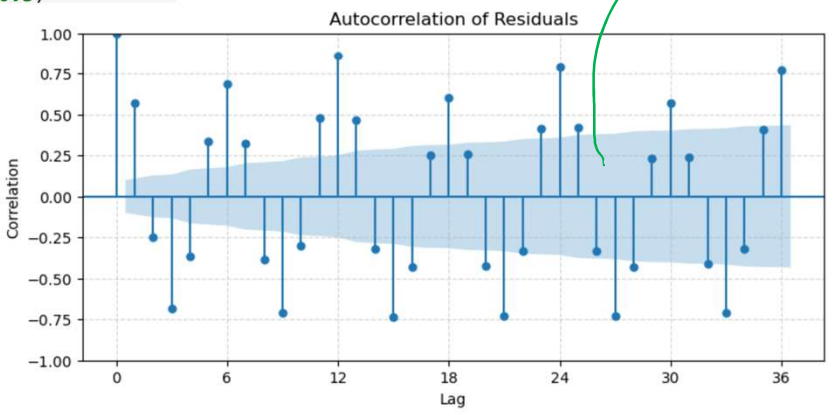
- Key observations:
  - Residuals are not purely random
  - Strong spikes at specific lags
  - Very high autocorrelation at lag 12 (~ 0.9)
    - Indicates remaining seasonal structure
    - Suggests 12-month cycle still present

  ⇒ Confirms incomplete modeling
- Possible improvement:
  - Use more flexible time series models
  - Refine seasonal component (e.g., may add more harmonics)

```python
# Using statsmodels
from statsmodels.tsa.stattools import acf

acf_values = acf(residuals['co2_val'], nlags=36)

fig, ax = plt.subplots(figsize=(9, 4))
ax.stem(range(len(acf_values)), np.abs(acf_values))

ax.set_xticks(range(0, 37, 6))
ax.set_title("Absolute Autocorrelation of Residuals")
ax.set_xlabel("Lag")
ax.set_ylabel("|Correlation|")
ax.grid(True, linestyle='--', alpha=0.5

plt.show();
```



```python
# Using statsmodels
from statsmodels.graphics.tsaplots import plot_acf

fig, ax = plt.subplots(figsize=(9, 4))
plot_acf(residuals['co2_val'], lags=36, ax=ax)

ax.set_xticks(range(0, 37, 6))
ax.set_title("Autocorrelation of Residuals")
ax.set_xlabel("Lag")
ax.set_ylabel("Correlation")
ax.grid(True, linestyle='--', alpha=0.5)

plt.show();
```



Exponential Smoothing

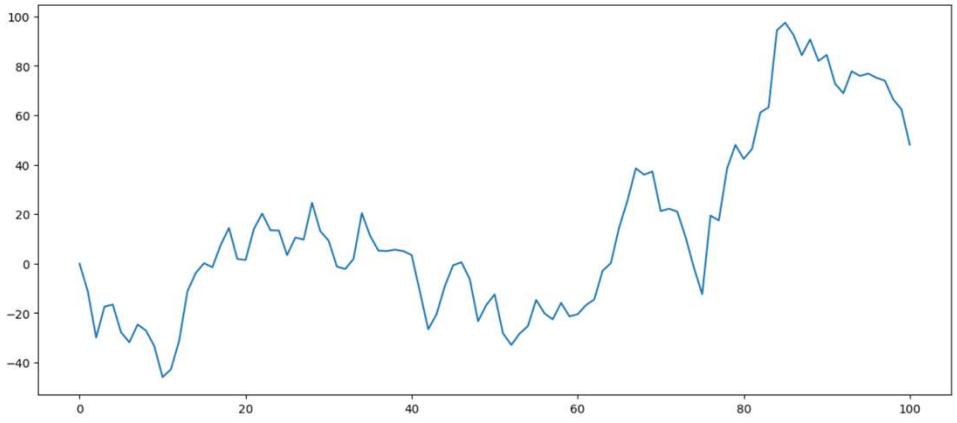
Forecasting random walks?

- Recall that a random walk is defined as

$$v_t = v_{t-1} + \epsilon_t$$

  where ϵ_t is white noise.
- Recall that we defined white noise as having:
  - Constant mean.
  - Constant variance.
  - Values are independently and identically distributed.
- A special but common case is the Gaussian noise: $\epsilon \sim \mathcal{N}(\mu, \sigma)$

```
plt.figure(figsize=(14, 6))

v = np.zeros(101)
for t in range(1, len(v)):
    v[t] = v[t-1] + 10 * np.random.normal(0, 1)

plt.plot(np.arange(101), v);
```



Random walks and white noise

- In a random walk, the change in values is merely the white noise:

$$y_t - y_{t-1} = \epsilon_t$$

- We know that we cannot predict white noise.
  - We don't know the distribution nor its parameters.
- In many cases, the best we can do is to assume that the future will be like the immediate past.
  - Causal factors will be the same in the short term.

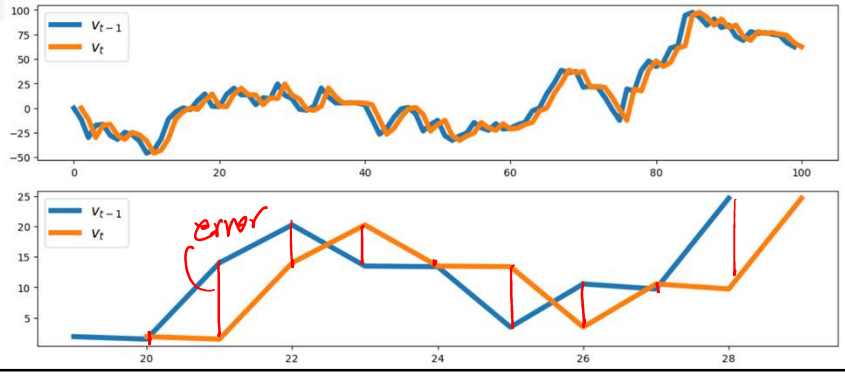
Predicting random walks

- The best guess for y_t is the previous value y_{t-1} :

$$\hat{y}_t = y_{t-1}$$

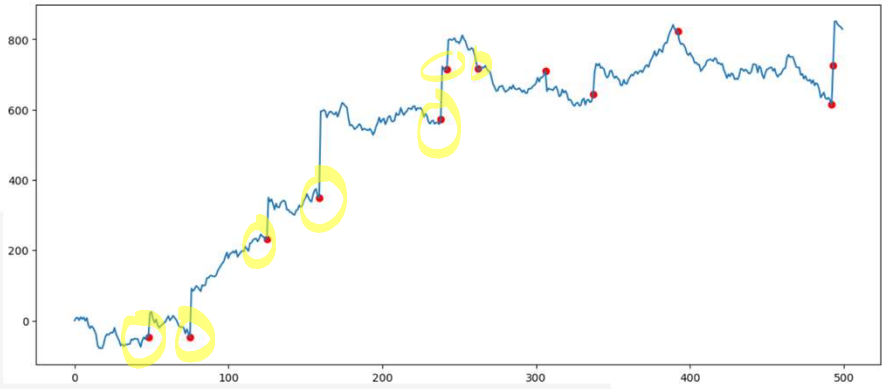
- This is the naïve forecast for random walks.
  - It serves as a baseline (benchmark) method.
- The forecast error is simply the noise term.

```
plt.figure(figsize=(14, 6))

x_axis = np.arange(0, 101)

plt.subplot(2,1,1)
plt.plot(x_axis[:-1], v[:100], label="$v_{t-1}$", linewidth=5)
plt.plot(x_axis[1:], v[:100], label="$v_t$", linewidth=5)
plt.legend(fontsize=14)

plt.subplot(2,1,2)
plt.plot(x_axis[19:29], v[19:29], label="$v_{t-1}$", linewidth=5)
plt.plot(x_axis[20:30], v[19:29], label="$v_{t}$", linewidth=5)
plt.legend(fontsize=14);
```



Forecasting using a sliding window

- Noise effect can be additive.
  - Multiple sources of noise can be compounded onto the signal.
  - For example,
    - in stock market, noise can be due to political instability, public health news, unforeseen weather events, etc.
    - IoT sensors can be subject to noise from ambient temperature, interference from external devices, natural interference, etc.

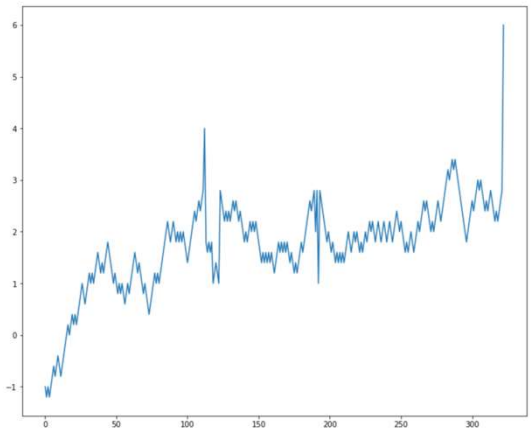



```
plt.figure(figsize=(14, 6))

v = np.zeros(500)
other_noise = 0
t_with_other_noise = []
for t in range(1, len(v)):
    v[t] = v[t-1] + 10 * np.random.normal(0, 1)
    if np.random.binomial(1, 0.02):
        other_noise = np.random.normal(100, 100)
        v[t] += other_noise
        t_with_other_noise.append(t-1)

plt.plot(v);
plt.scatter(t_with_other_noise, np.array(v)[t_with_other_noise], color="red")
```

$v_t = v_{t-1} + \varepsilon_t$

Forecasting using a sliding window (cont'd.)

- In some cases, the effect of some sources of noise can be short-lived.
  - Cause major spikes in the data that can bias predictions.
  - Time series regresses to its original value after the noise subsides.



Forecasting using a sliding window (cont'd.)

- Outliers can significantly affect the prediction.
- To know the past, we need to abstract the noise in the data, i.e., smooth the data.
- We can smooth the prediction by using the average of "some" n previous values.

$$v_{t+1} = \frac{1}{n}\sum_{i=0}^{n-1} v_{t-i}$$

$v_t, v_{t-1}, \cdots, v_{t-n+1}$

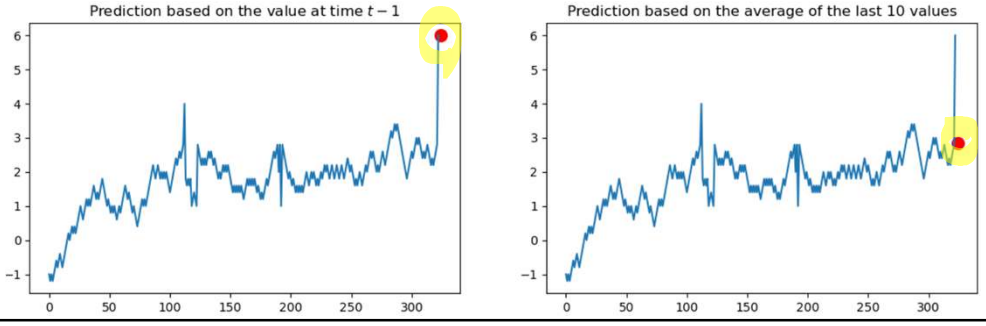
n past obs.

```
random_ts = pd.read_csv("data/rand_ts.csv", header=None, names=['val'])
random_ts
```

	val
0	-1.0
1	-1.2
2	-1.0
3	-1.2
4	-1.0
...	...
318	2.2
319	2.4
320	2.6
321	2.8
322	6.0

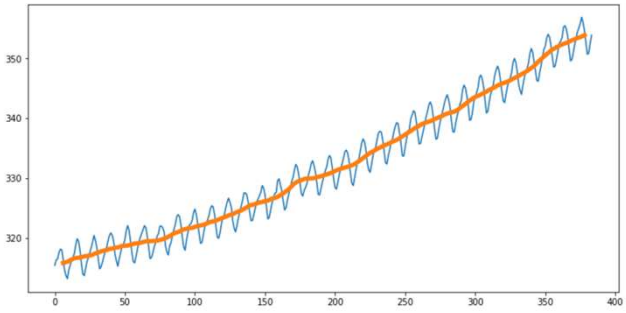
323 rows × 1 columns

```
plt.figure(figsize=(14, 4))

pred = random_ts[-10:].mean()

plt.subplot(1,2,1)
plt.plot(random_ts)
plt.scatter(len(random_ts)+1, random_ts.val.iloc[-1], color='r', s=100)
plt.title("Prediction based on the value at time $t-1$", fontsize=12)

plt.subplot(1,2,2)
plt.plot(random_ts)
plt.scatter(len(random_ts)+1, pred, color='r', s=100)
plt.title("Prediction based on the average of the last 10 values", fontsize=12);
```



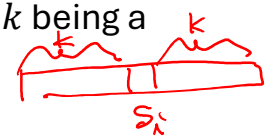
Smoothing data

- This strategy is the same as the one we used for computing the rolling average for the CO_2 data.
  - Smoothing the curve prior to finding the power law parameter that best fits it.



Smoothing using a running or moving average

- For every value x_i, we compute a new smoothed score value s_i.
  - I.e., compute x_i in light of its neighbors' values.
  - Uses the sliding window strategies discussed before.
- Example 1: For some window of size $2k + 1$, with k being a positive value, we compute s_i as:

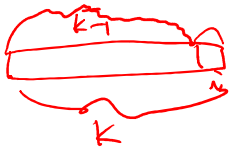


$$s_i = \frac{1}{2k+1} \sum_{j=-k}^{k} x_{i+j}$$

  - Ex. for $k = 3$, the window size is $2k + 1 = 7$, and computing the smoothed score s_{11} at position x_{11} amounts to averaging the score for positions: $x_8, x_9, x_{10}, x_{11}, x_{12}, x_{13}, x_{14}$

Smoothing using a running or moving average (cont'd.)

- Example 2: For some window of size k, we compute s_i as:

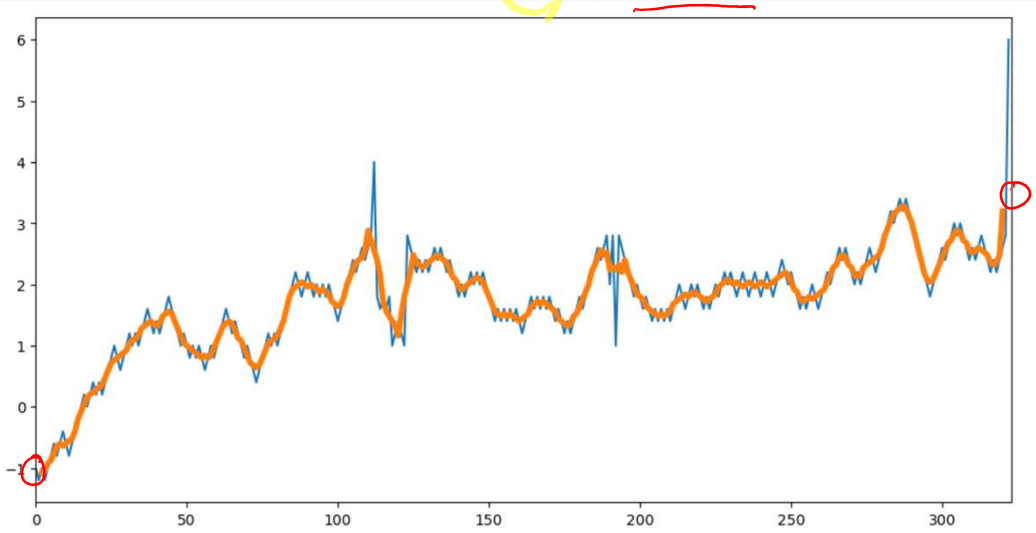


$$s_i = \frac{1}{k} \sum_{j=0}^{k-1} x_{i-j}$$

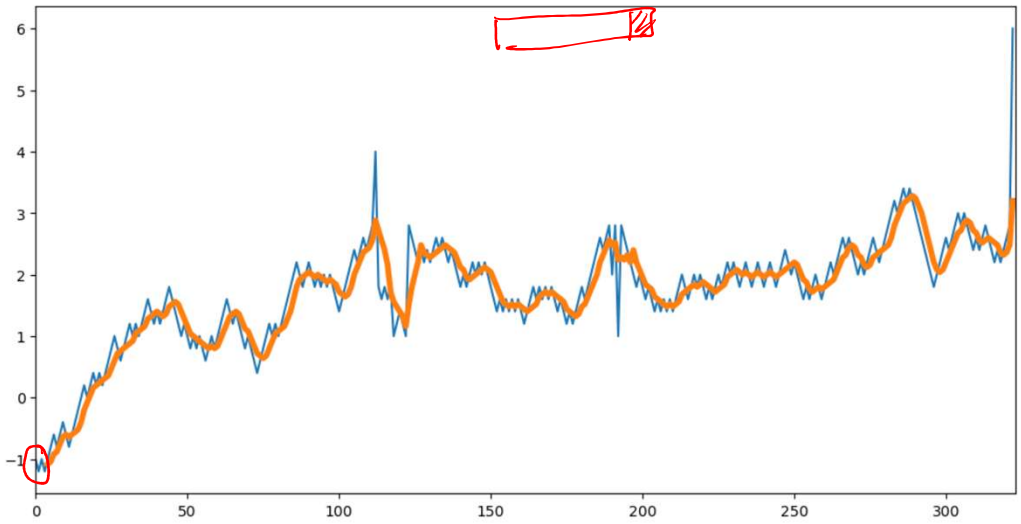
  - Ex. for $k = 4$, computing the smoothed score s_{11} at position x_{11} amounts to averaging the score for positions:

$$x_8, x_9, x_{10}, x_{11}$$

```
plt.figure(figsize=(12, 6))
plt.xlim(0, len(random_ts))
plt.plot(random_ts)
plt.plot(random_ts['val'].rolling(window=5, center=True).mean(), linewidth=4);
```



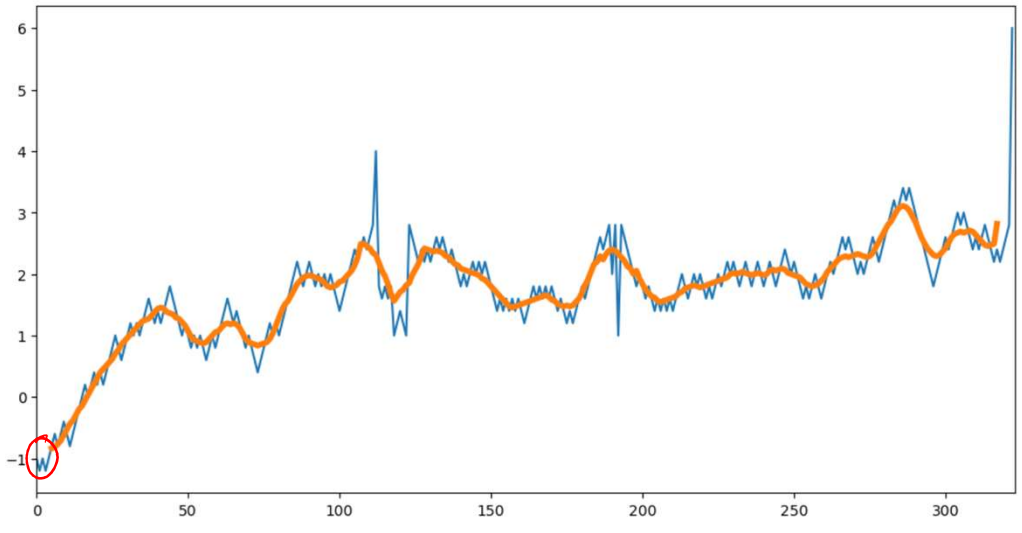

```
plt.figure(figsize=(12, 6))
plt.xlim(0, len(random_ts))
plt.plot(random_ts)
plt.plot(random_ts['val'].rolling(window=5).mean(), linewidth=4);
```



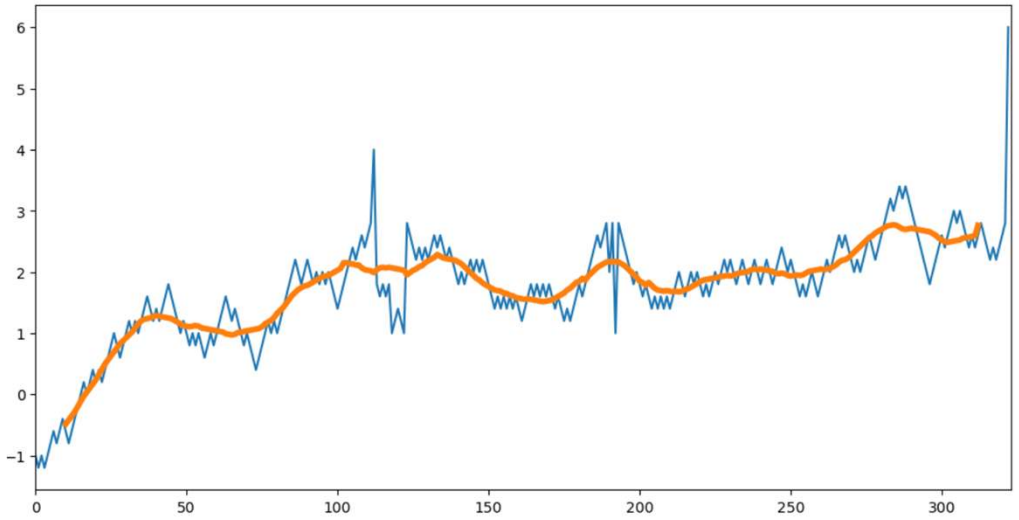
Effect of the window size

- The rolling window smooths the data, but the curve is still bumpy around large spikes.
  - Large values continue to influence the average while they remain within the window.
- Sudden jumps in the moving average value when a relatively large observation (a spike) comes into or falls out of the moving average window.
- We can further smooth the curve by choosing a larger window size.
  - More values to dampen the effect of larger values.

```
plt.figure(figsize=(12, 6))
plt.xlim(0, len(random_ts))
plt.plot(random_ts)
plt.plot(random_ts['val'].rolling(window=11, center=True).mean(), linewidth=4);
```

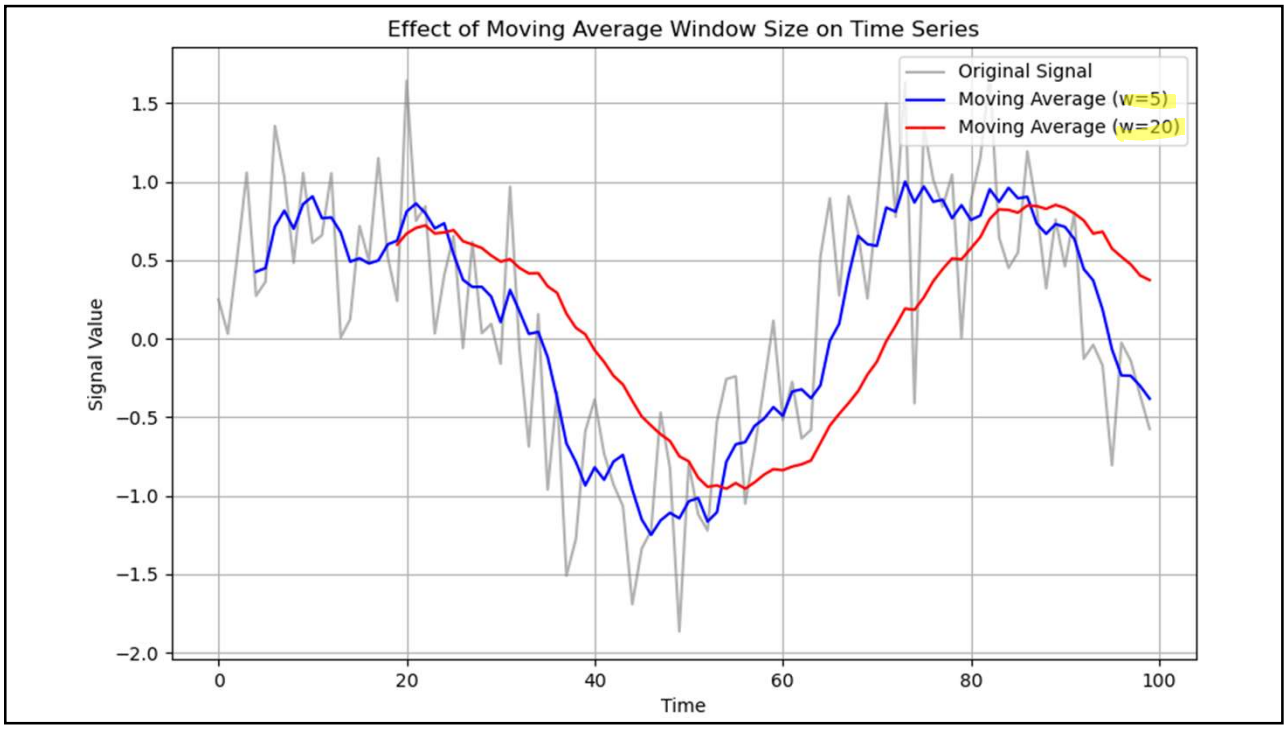


```
plt.figure(figsize=(12, 6))
plt.xlim(0, len(random_ts))
plt.plot(random_ts)
plt.plot(random_ts['val'].rolling(window=21, center=True).mean(), linewidth=4);
```



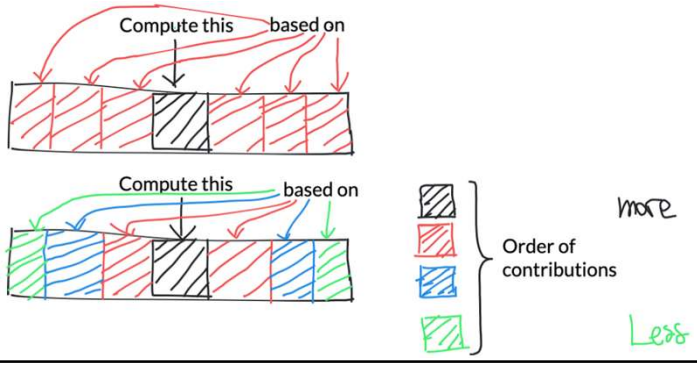
Handling outliers

- Increasing the window size to compensate for occasional spikes might result in not capturing the true essence of the data.
  - Excessively smooths the curve.
    - Unnecessarily levels out even legitimate changes in the data.
    - Introduces a delay (“lag”) in the signal.
- Reducing the window size to respond to genuine changes might cause the data to be too closely fitted.
  - Compromises the data smoothing capability.
  - Leads to abrupt changes that do not accurately represent the underlying data trends.



Handling outliers (cont'd.)

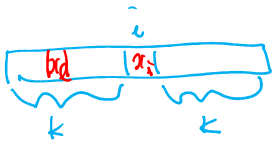
- We can assign the same preset values to specific indices.
- We can keep a large window size but, for instance, weigh the values in the window differentially.
  - The points close to window boundaries contribute less than the points near the center.



Using a Gaussian for computing the contributions

- We can use more sophisticated versions that assign custom weights. E.g.,

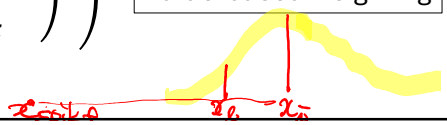
$$s_i = \sum_{j=-k}^{k} w_{i+j} x_{i+j}$$



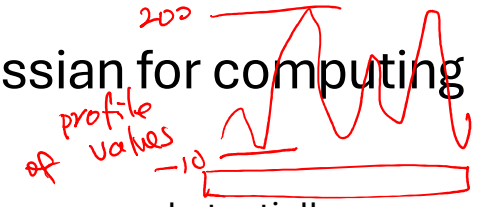
where weight w_l for a point x_l can be determined using, for example, a Gaussian:

$$f(x_i, \sigma_x) = \frac{1}{\sqrt{2\pi}\sigma_x} \exp\left(-\frac{1}{2}\left(\frac{x_l - x_i}{\sigma_x}\right)^2\right)$$

Value-based weighting



Challenges of using a Gaussian for computing the contributions

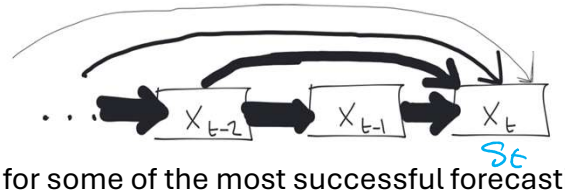


- The range of values in each window can vary substantially.
- For each window, we need to dynamically scale σ so that the likelihoods (weights) are not too small.

- Easy to implement but not ideal:
  - The value-based weighted moving average can be computationally intensive to evaluate.
    - For each window, we need to determine a specific σ and compute the likelihoods for all the points in the window.
  - We lose data at the beginning and the end of data due to window constraints.
- No easy way to incorporate information about the trend or seasonality.

Exponential smoothing

- Exponential smoothing mitigates the shortcomings of the rolling window and weighted moving averages.
- Intuition: Instead of using simply a subset of the data, we will use all past observations.
  - The contributions (weights) associated with each point decays exponentially as the observations "get older".



- This simple concept is the basis for some of the most successful forecasting methods.
  - One such method is exponential smoothing.

Forms of exponential smoothing

- Different types of smoothing capture different types of signals.

Type of Smoothing	Data type
Simple Exponential Smoothing	Neither trend nor seasonality
Double Exponential Smoothing	Trend but no seasonality
Triple Exponential Smoothing	Trend and seasonality

Single exponential smoothing

- Computes smoothed data s_t at time t using the observed value v_t and smoothed value from previous time step (s_{t-1})

$$s_t = \alpha \cdot v_t + (1-\alpha)s_{t-1} \quad \text{where } 0 \le \alpha \le 1$$

  - s_t: smoothed value at time step t
  - v_t: actual (unsmoothed) value at time step t
  - s_{t-1}: smoothed value at time step $t-1$
- Note that at:
  - $\alpha = 0$, $s_t = s_{t-1}$; we are only retaining the past value. $\therefore s_t = s_{t-1} = \cdots = s_1 = v_1$
  - $\alpha = 1$, $s_t = v_t$; we are only retaining current values

$s_{t-1} = v_{t-1}$

Single exponential smoothing: Remarks

How is this exponential?

It turns out that:

$$\begin{aligned}
s_t &= \alpha \cdot v_t + (1-\alpha)s_{t-1} \\
&= \alpha \cdot v_t + (1-\alpha)(\alpha \cdot v_{t-1} + (1-\alpha)s_{t-2}) \\
&= \alpha \cdot v_t + \alpha(1-\alpha)v_{t-1} + (1-\alpha)^2 s_{t-2} \\
&= \alpha[v_t + (1-\alpha)v_{t-1}] + (1-\alpha)^2 s_{t-2} \\
&= \alpha[v_t + (1-\alpha)v_{t-1}] + (1-\alpha)^2(\alpha \cdot v_{t-2} + (1-\alpha)s_{t-3}) \\
&= \alpha[v_t + (1-\alpha)v_{t-1} + (1-\alpha)^2 v_{t-2}] + (1-\alpha)^3 s_{t-3} \\
&\vdots \\
&= \alpha[v_t + (1-\alpha)v_{t-1} + (1-\alpha)^2 v_{t-2} + \cdots + (1-\alpha)^{t-1} v_1] + (1-\alpha)^t v_0
\end{aligned}$$

$s_t = \alpha v_t + \alpha(1-\alpha)v_{t-1} + \alpha(1-\alpha)^2 v_{t-2} + \cdots + \alpha(1-\alpha)^t v_0$

```
alpha = 0.8
t = 10
[round((1-alpha)**i, 4) for i in range(t)]
```

```
[1.0, 0.2, 0.04, 0.008, 0.0016, 0.0003, 0.0001, 0.0, 0.0, 0.0]
```

```
alpha = 0.2
t = 10
[round((1-alpha)**i, 4) for i in range(t)]
```

```
[1.0, 0.8, 0.64, 0.512, 0.4096, 0.3277, 0.2621, 0.2097, 0.1678, 0.1342]
```

$S_t = \alpha v_t + (1-\alpha) S_{t-1}$

Single exponential smoothing: Remarks (cont'd.)

$$s_t = \alpha[v_t + (1-\alpha)v_{t-1} + (1-\alpha)^2 v_{t-2} + \cdots + (1-\alpha)^{t-1} v_1] + (1-\alpha)^t v_0$$

- All the previous values contribute to smoothed value of x_i but their contribution gets increasingly small.
  - The contribution of past values is controlled by the decay of the parameter α.
- This geometric progression (geometric series) is the discrete version of an exponential function.
  - A geometric sequence is discrete while an exponential function is continuous.
    - This is where the name for this smoothing method originated.

Single exponential smoothing: Remarks (cont'd.)

- Given that we have smoothed our signal incorporating all previous observations, we can use s_t in forecasting.
- Our approach is going to be simple:

$$F_{t+1} = s_t$$

- Our data is more resilient to outliers.
- The prediction takes into account weighted contributions of previous time points.

Single exponential smoothing as a function of the forecasting error

- Note that the forecast F_{t+1} at time $t+1$ is similar to the random walk algorithm used earlier:

$$\begin{aligned} s_t &= \alpha \cdot v_t + (1-\alpha)s_{t-1} \\ &= \alpha \cdot v_t + s_{t-1} - \alpha \cdot s_{t-1} \\ &= s_{t-1} + \alpha(v_t - s_{t-1}) \end{aligned}$$

$F_{t+1} = S_t$

$F_t = S_{t-1}$

  - Since $F_{t+1} = s_t$, and $F_t = s_{t-1}$,

$$F_{t+1} = F_t + \alpha(v_t - F_t) = F_t + \alpha E$$

    where αE is here a fraction of E (the difference between forecasted and observed values) in forecasting.

error

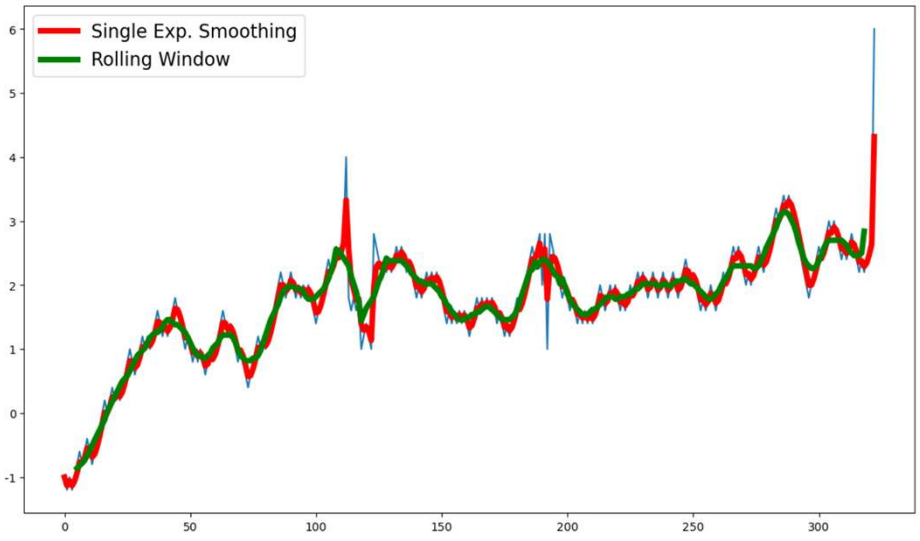
- This means that the forecast F_{t+1} is simply the forecast at time t (F_t) padded with a fraction of the error in prediction F_t.

Single exponential smoothing with `Pandas.Series`

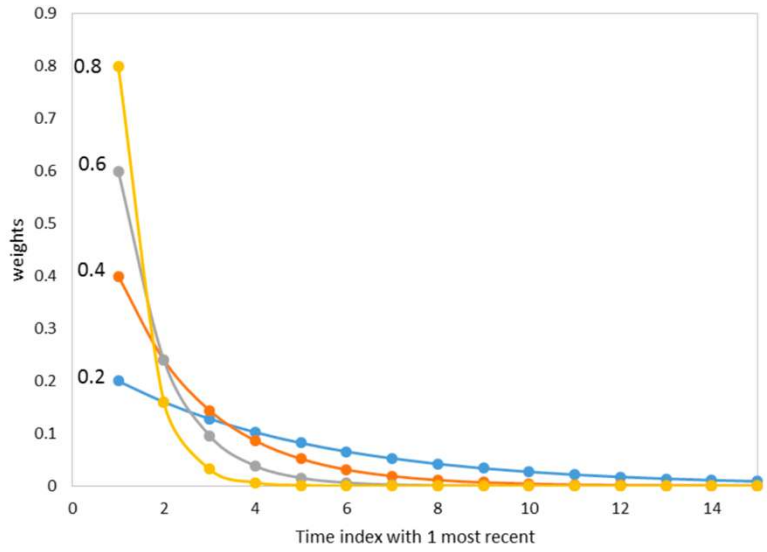
- The Series object has an `ewm` (exponentially weighted moving).
- Note that single exponential smoothing is simply a complex weighted average.
- We compute the new values by using the mean function:

```
random_ts['val'].ewm(alpha=0.5).mean()
```

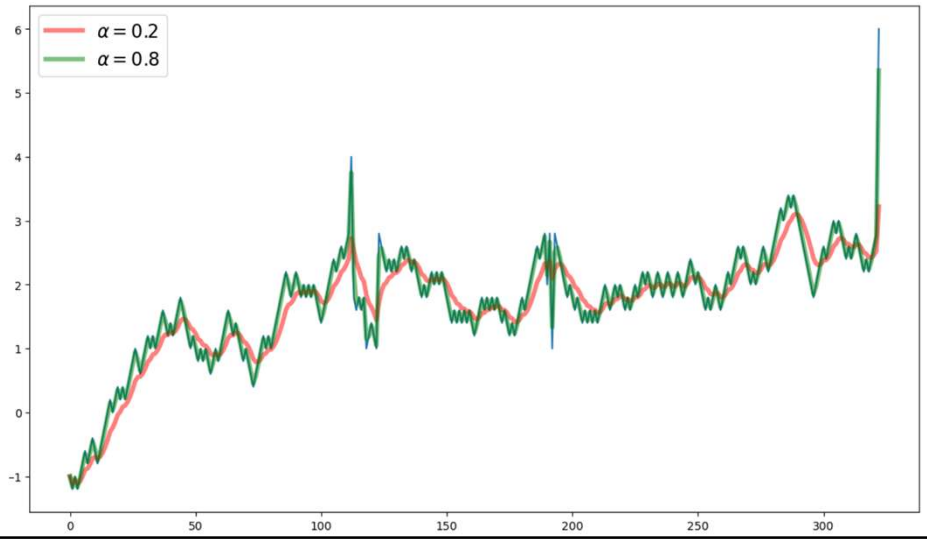
```
plt.figure(figsize=(14, 8))
plt.plot(random_ts)
plt.plot(random_ts['val'].ewm(alpha=0.5).mean(), color='r', lw=5, label="Single Exp. Smoothing")
plt.plot(random_ts['val'].rolling(window=10, center=True).mean(), color='g', lw=5, label="Rolling Window")
plt.legend(fontsize=16);
```



Impact of parameter α




```
plt.figure(figsize=(14, 8))
plt.plot(random_ts)
plt.plot(random_ts['val'].ewm(alpha=0.2).mean(), color='r', lw=4, alpha=0.5, label=r"$\alpha = 0.2$")
plt.plot(random_ts['val'].ewm(alpha=0.8).mean(), color='g', lw=4, alpha=0.5, label=r"$\alpha = 0.8$")
plt.legend(fontsize=16);
```



Double exponential smoothing

- It would be unwise to discard the trend information if one exists.
- Double exponential smoothing add the information about the trend.
  - A "non-parametric equivalent" for identifying the equation that best fits the trend.
- We will include the trend in our forecast.

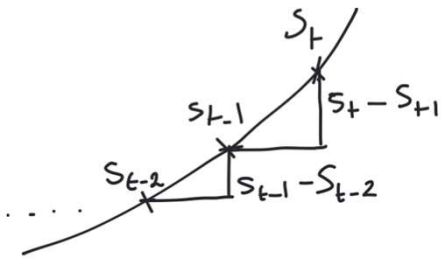
Accounting for the trend

- It helps to think of the trend as the exponentially weighted changes since $t = 0$.
  - The most recent change gets weight of β.
- We can infer the trend r_t using the same approach used to compute s_t:

$$r_t = \beta(s_t - s_{t-1}) + (1 - \beta)r_{t-1}$$

- This equation uses a trend coefficient β which acts in a similar way as α on signal.
  - Recall that the assumption here is that there is no seasonality.

$S_t = \alpha V_t + (1-\alpha)S_{t-1}$



Double exponential smoothing (cont'd.)

- Similar to the equation used in single exponential smoothing.
  - We add the effect of the trend to s_t.

$$s_t = \alpha \cdot v_t + (1 - \alpha)(s_{t-1} + r_{t-1})$$

- Forecasting:
  - Future value is estimated by using the last smoothed value and adding, at each predicted timestep, the smoothed trend to it.

$$F_{t+1} = s_t + r_t$$